

O `ngModel` no Angular é uma diretiva para **two-way data binding** (vinculação de dados bidirecional), sincronizando em tempo real o valor de um campo de entrada (`input`, `select`, `textarea`) com uma variável no componente. Ele é essencial em formulários baseados em templates para capturar e exibir dados. [1, 2, 3]

Assista a este vídeo para ver como usar o `[(ngModel)]` em Angular:

[\[\(ngModel\)\] em Angular | Torne-se um Programador](#), YouTube · Danilo Aparecido - Torne-se um programador · 2020 M12 18

Exemplo Prático de uso do `ngModel`

1. Importar o `FormsModule`

Para usar o `ngModel`, você deve importar o `FormsModule` no seu módulo principal (`app.module.ts`) ou diretamente no componente (se for um componente Standalone). [4, 5]

```
import { FormsModule } from '@angular/forms';
```

```
@Component({  
  // ...  
  standalone: true,  
  imports: [FormsModule, /* outros imports */],  
})  
export class AppComponent { ... }
```

2. Criar a variável no Componente (`app.component.ts`)

```
import { Component } from '@angular/core';  
import { FormsModule } from '@angular/forms';
```

```
@Component({  
  selector: 'app-root',  
  standalone: true,  
  imports: [FormsModule],  
  templateUrl: './app.component.html',  
  styleUrls: ['./app.component.css']  
})  
export class AppComponent {  
  // A variável que será sincronizada  
  nomeUsuario: string = '';
```

```
}
```

3. Usar [(ngModel)] no Template (app.component.html) [6]

```
<input type="text" [(ngModel)]="nomeUsuario" placeholder="Digite seu nome">
```

```
<p>Olá, {{ nomeUsuario }}!</p>
```

Pontos Chave

- **Sintaxe [(ngModel)]:** A sintaxe "banana in a box" [()] permite que a mudança no input altere a variável, e a mudança na variável altere o input.
- **Requisito de name:** Ao usar [(ngModel)] dentro de um formulário (<form>), é obrigatório definir o atributo name no elemento input.
- **Uso com Signals:** No Angular moderno, você pode vincular sinais (Signals) diretamente usando [(ngModel)] para reatividade. [1, 3, 5]

O uso de ngModel com **Signals** no Angular é surpreendentemente simples. Você pode vincular um WritableSignal diretamente à diretiva [(ngModel)] sem precisar chamar a função do signal no template. [1, 2]

Assista a este vídeo para entender como as APIs de Model do Angular simplificam o two-way data binding com signals: [1]

[Angular 17 - Model Inputs - Two way data binding](#), YouTube · Code with Ahsan · 2024 M03 6

Exemplo Direto com signal()

Se você tem um sinal comum (WritableSignal), basta passar a referência dele para o [(ngModel)]. Note que **não** se usa parênteses () dentro do colchete do ngModel. [3]

Componente (app.component.ts):

```
import { Component, signal } from '@angular/core';
```

```
import { FormsModule } from '@angular/forms';
```

```
@Component({
```

```
  selector: 'app-root',
```

```
  standalone: true,
```

```
  imports: [FormsModule],
```

```
  template: `
```

```
    <!-- Referência ao signal SEM parênteses -->
```

```
    <input [(ngModel)]="nome" placeholder="Digite algo...">
```

<!-- Leitura do signal COM parênteses -->

<p>O nome é: {{ nome() }}</p>

}}

export class App {

nome = signal('Angular');

}

Usando a API model() (Angular 17.2+) [1, 4]

Para componentes que recebem dados de um pai e precisam devolvê-los (Two-way binding), o Angular introduziu o model(). Ele cria um sinal que funciona como um par de @Input e @Output. [5]

1. **Componente Filho:** Define a propriedade usando model().
2. export class ContadorComponent {
3. valor = model(0); *// Cria um Signal de entrada/saída*
4. }
5. **Componente Pai:** Consome usando a sintaxe de "banana na caixa".
6. <app-contador [(valor)]="meuSignalPai" />

[6]

Principais Diferenças

- **Referência vs Valor:** No [(ngModel)]="nome", você passa a **referência** do sinal para que o Angular saiba onde gravar o novo valor.
- **Exibição:** Para mostrar o valor no texto (interpolação), você continua chamando o sinal como uma função: {{ nome() }}.
- **Alternativa Explícita:** Se preferir não usar o binding bidirecional automático, você pode separar: [ngModel]="nome()" (para ler) e (ngModelChange)="nome.set(\$event)" (para escrever). [2, 7, 8, 9, 10]

Você gostaria de ver como integrar esses **Signals** com validações de formulário ou em **formulários reativos**?